

Q&A — Logistic Regression with Classification Report & ROC Curve

Dataset: `data/default.csv`

Task: Predict whether a customer will `default` (Yes/No) from `balance` and `income`.

Full Example — End to End

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    classification_report,
    confusion_matrix,
    roc_curve,
    auc,
    ConfusionMatrixDisplay
)

# — 1. Load & prepare data —————
df = pd.read_csv('data/default.csv')
df['default_yes'] = (df['default'] == 'Yes').astype(int)

X = df[['balance', 'income']].values
y = df['default_yes'].values

# — 2. Train / test split —————
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# — 3. Fit logistic regression —————
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)          # predicted class labels (0 or 1)
y_pred_prob = model.predict_proba(X_test)[:, 1]  # P(default=1)

# — 4. Classification Report —————
report_dict = classification_report(y_test, y_pred, output_dict=True)
report_str = classification_report(y_test, y_pred,
                                   target_names=['Class 0 (No Default)',
                                                'Class 1 (Default)'])
print(report_str)
```

Sample output:

	precision	recall	f1-score	support
Class 0 (No Default)	0.97	1.00	0.99	2893
Class 1 (Default)	0.87	0.38	0.53	97
accuracy			0.97	2990
macro avg	0.92	0.69	0.76	2990
weighted avg	0.97	0.97	0.97	2990

Extract & Display Class 0 and Class 1 Metrics

```
# — 5. Print Class 0 and Class 1 metrics individually
for cls_key, cls_label in [ ('0', 'Class 0 – No Default'), ('1', 'Class 1 – Default') ]:
    p = report_dict[cls_key]['precision']
    r = report_dict[cls_key]['recall']
    f1 = report_dict[cls_key]['f1-score']
    s = report_dict[cls_key]['support']
    print(f"{cls_label}")
    print(f" Precision : {p:.4f}")
    print(f" Recall    : {r:.4f}")
    print(f" F1-Score   : {f1:.4f}")
    print(f" Support    : {int(s)}")
    print()
```

Sample output:

```
Class 0 – No Default
Precision : 0.9736
Recall    : 0.9997
F1-Score  : 0.9865
Support   : 2893

Class 1 – Default
Precision : 0.8750
Recall    : 0.3814
F1-Score  : 0.5313
Support   : 97
```

Plot Classification Report as a Bar Chart

```
# — 6. Visualise classification report —————
metrics = ['precision', 'recall', 'f1-score']
cls0_vals = [report_dict['0'][m] for m in metrics]
cls1_vals = [report_dict['1'][m] for m in metrics]

x = np.arange(len(metrics))
width = 0.35

fig, ax = plt.subplots(figsize=(8, 5))
bars0 = ax.bar(x - width/2, cls0_vals, width, label='Class 0 (No Default)',
               color='steelblue')
bars1 = ax.bar(x + width/2, cls1_vals, width, label='Class 1 (Default)',
               color='salmon')

ax.set_xticks(x)
ax.set_xticklabels(['Precision', 'Recall', 'F1-Score'])
ax.set_ylim(0, 1.1)
ax.set_ylabel('Score')
ax.set_title('Classification Report – Logistic Regression')
ax.legend()

# Add value labels on bars
for bar in bars0 + bars1:
    ax.text(bar.get_x() + bar.get_width() / 2,
            bar.get_height() + 0.02,
            f'{bar.get_height():.2f}',
            ha='center', va='bottom', fontsize=9)

plt.tight_layout()
plt.show()
```

Confusion Matrix

```
# — 7. Confusion matrix —————
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:")
print(cm)
# [[TN  FP]
#  [FN  TP]]

disp = ConfusionMatrixDisplay(confusion_matrix=cm,
                              display_labels=['No Default', 'Default'])
disp.plot(cmap='Blues')
plt.title('Confusion Matrix – Logistic Regression')
plt.show()
```

ROC Curve with AUC

```
# — 8. ROC Curve —————
fpr, tpr, thresholds = roc_curve(y_test, y_pred_prob)
roc_auc = auc(fpr, tpr)

plt.figure(figsize=(7, 6))
plt.plot(fpr, tpr, color='darkorange', lw=2,
         label=f'ROC Curve (AUC = {roc_auc:.4f})')
plt.plot([0, 1], [0, 1], color='navy', lw=1.5, linestyle='--',
         label='Random Classifier (AUC = 0.50)')
plt.fill_between(fpr, tpr, alpha=0.1, color='darkorange') # shade area under
curve

plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate (1 - Specificity)', fontsize=12)
plt.ylabel('True Positive Rate (Sensitivity / Recall)', fontsize=12)
plt.title('ROC Curve – Logistic Regression (Default Dataset)', fontsize=13)
plt.legend(loc='lower right', fontsize=11)
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()

print(f"AUC = {roc_auc:.4f}")
```

All Together — Copy-Paste Ready

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import (classification_report, confusion_matrix,
                             ConfusionMatrixDisplay, roc_curve, auc)

# Data
df = pd.read_csv('data/default.csv')
df['default_yes'] = (df['default'] == 'Yes').astype(int)
X = df[['balance', 'income']].values
y = df['default_yes'].values

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
                                                    random_state=42)

# Model
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
y_pred_prob = model.predict_proba(X_test)[: , 1]
```

```

# Classification report
report_dict = classification_report(y_test, y_pred, output_dict=True)
print(classification_report(y_test, y_pred,
                           target_names=['Class 0 (No Default)', 'Class 1 (Default)']))

# Per-class metrics
for cls_key, cls_label in [('0', 'Class 0 - No Default'), ('1', 'Class 1 - Default')]:
    p = report_dict[cls_key]['precision']
    r = report_dict[cls_key]['recall']
    f1 = report_dict[cls_key]['f1-score']
    s = report_dict[cls_key]['support']
    print(f"{cls_label} | Precision: {p:.4f} Recall: {r:.4f} F1: {f1:.4f} Support: {int(s)}")

# Bar chart of classification report
metrics = ['precision', 'recall', 'f1-score']
cls0_vals = [report_dict['0'][m] for m in metrics]
cls1_vals = [report_dict['1'][m] for m in metrics]
x, width = np.arange(3), 0.35

fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# - Plot 1: Classification report bar chart
ax = axes[0]
b0 = ax.bar(x - width/2, cls0_vals, width, label='Class 0', color='steelblue')
b1 = ax.bar(x + width/2, cls1_vals, width, label='Class 1', color='salmon')
ax.set_xticks(x); ax.set_xticklabels(['Precision', 'Recall', 'F1-Score'])
ax.set_ylim(0, 1.15); ax.set_ylabel('Score')
ax.set_title('Classification Report'); ax.legend()
for bar in list(b0) + list(b1):
    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.02,
            f'{bar.get_height():.2f}', ha='center', va='bottom', fontsize=9)

# - Plot 2: Confusion matrix
ax = axes[1]
cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm,
                              display_labels=['No Default', 'Default'])
disp.plot(ax=ax, cmap='Blues', colorbar=False)
ax.set_title('Confusion Matrix')

# - Plot 3: ROC Curve
ax = axes[2]
fpr, tpr, _ = roc_curve(y_test, y_pred_prob)
roc_auc = auc(fpr, tpr)
ax.plot(fpr, tpr, color='darkorange', lw=2, label=f'AUC = {roc_auc:.4f}')
ax.plot([0, 1], [0, 1], 'navy', lw=1.5, linestyle='--', label='Random (AUC = 0.50)')
ax.fill_between(fpr, tpr, alpha=0.1, color='darkorange')
ax.set_xlim([0, 1]); ax.set_ylim([0, 1.05])
ax.set_xlabel('False Positive Rate'); ax.set_ylabel('True Positive Rate')
ax.set_title('ROC Curve'); ax.legend(loc='lower right'); ax.grid(alpha=0.3)

```

```
plt.suptitle('Logistic Regression – Default Dataset', fontsize=14, y=1.01)
plt.tight_layout()
plt.show()
```

Metric Definitions (Quick Reference)

Metric	Formula	Meaning
Precision	$TP / (TP + FP)$	Of all predicted positive, how many are actually positive
Recall	$TP / (TP + FN)$	Of all actual positives, how many did we correctly catch
F1-Score	$2 \times (P \times R) / (P + R)$	Harmonic mean of precision and recall
Support	Count of actual class	Number of true instances in that class
AUC	Area under ROC curve	1.0 = perfect, 0.5 = random, < 0.5 = worse than random
Sensitivity	= Recall = TPR	True Positive Rate
Specificity	$TN / (TN + FP)$	True Negative Rate = $1 - FPR$